

PILLARS OF OOP'S

1) Encapsulation

- Both data and Behaviour should be together (encapsulated) Within an object.
- We should not provide unrestricted access of data. Means should provide restricted access of data.

Access modifiers

Access Modifiers controls who have access of variable, method, constructor, class?

4 Types of Access Modifiers

- **(public)** Access Modifier—> Anyone can access, (most unrestricted).
- **(protected)** Access Modifier—> Only same package access + Inherited class (child class) access.
- **(default)** Access Modifier—> Anyone can access belonging to same package. Ye defaultly lag jata ha.
- **(private)** Access Modifier—> no one can access out side of the (class).

```
public class Encapsulation {
    public static void main(String[] args) {
        BankAccount ba = new BankAccount ();
        ba.deposit(500);
        ba.withdraw(200);
        System.out.println(ba.getBalance());
    }
}

class BankAccount{
    private double balance;// "private" only can be accessed inside the class(BankAccount);

    public void deposit (int amount){// 'public' anyone can access.
        balance += amount;
    }
    public void withdraw (int amount){
        balance -= amount;
    }
    // getters / setters
    public double getBalance (){
        return balance;
    }
}

class Student{
    private String name;// private Access modifier.
    private int age;
    private int rollNumber;
    private String college;

    Student(String name, int age, int rollNumber, String college){
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
        this.college = college;
    }
    // getters / setters---> inka kaam proper validations lagana ha taki koi bhi unnecessary values
    na daale.
    public String getCollege(){ // public access modifier.
        return name;
    }
}
```

```

public void setCollege(String college) {
    // validations ---> college should be real.
    this.college = college;
}
}

```

“Package”

Note: “Package” directly means ki kisi bhi “.java” vali file ko kisi particular folder me dalna.

- Kisi bhi new file me agar kisi package ki file ki class ko dalna ha to uske liye muj (**import packageName.className**) dalna hoga.
- Or agar muj kisi package ki sabhi class ko ek sath add krna ha to top me (**import packageName.***) dalna hoga.
- And these above packages are known as **USER DEFINED PACKAGES**.
- Whereas java has **In-built packages** also in their LIBRARIES, for **example**→
java.util
java.lang
java.io

2) Inheritance

- It shows the relations between two classes of single package’s file.
- It shows “**is-a**” relationship, Example: Vechile→ cars, bike, etc.
- Showing that car “**is-a**” vehicle.
- Programatically it defined by→ (**extends**) keyword.
- It gives relief of **code reusability**.
- It helps to support polymorphism.

Types of inheritance are shown below.

1) **Simple inheritance**→ One Parent class→One child class.

```

class Student { // parent class.
    String name;
    int age;

    void markAttendance() {
        System.out.println("Attendance Marked");
    }
}

class EngineeringStudent extends Student{ // child class.
    //no need to add 'markAttendance()' because it is a child class of 'Student'.
    void attendLab(){
        System.out.println("Lab attended");
    }
}

```

2) **Multi-level inheritance**→ (parent→ child→child’s child.).

```

class Student { // parent class.
    String name;
    int age;

    void markAttendance() {
        System.out.println("Attendance Marked");
    }
}

class EngineeringStudent extends Student{ // child class.
    void attendLab(){
        System.out.println("Lab attended");
    }
}

```

```

    }
}
class CSEEngineeringStudent extends EngineeringStudent{ // child's child
}

```

3) Hierarchical inheritance → One parent class → child class 01, child class 02.

```

class Student { // parent class.
    String name;
    int age;

    void markAttendance() {
        System.out.println("Attendance Marked");
    }
}

class EngineeringStudent extends Student{ // child class.
    void attendLab() {
        System.out.println("Lab attended");
    }
}

class MedicalStudent extends Student{ // another child class
}

```

4) Multiple inheritance → Parent-1 class, Parent-2 class → common child class.

- Not supported in java, due to diamond problem.

Super keyword

- It is used to access parent class's **variable**.

```

class Student { // parent class.
    String name;
    int age;
    int rollNumber;
    int x;
}

class EngineeringStudent extends Student{ // child class.
    String college;
    int x;
    void Print() {
        System.out.println(super.name + ", " + super.age + ", " + super.rollNumber + ", " + super.x);
        //calling the variables of parent class.
    }
}

```

-
- It can call to the **methods** of parent class.

```

class Student { // parent class.
    String name;
    int age;
    int rollNumber;

    void Print() {
        System.out.println(name + ", " + age + ", " + rollNumber );
    }
}

class EngineeringStudent extends Student{ // child class.
    String college;

    void Print() {
        super.Print();
        System.out.println(college);
    }
}

```

- Can call parent class **constructor**, by→ **super.()**; // almost similar to methods calling.